15.4
Event-Driven Architecture

Async communication with Kafka

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

What is Event-Driven Architecture?

Instead of Service A calling Service B directly (synchronous), Service A publishes an event ('something happened') and walks away. Service B (and C and D) can listen and react at their own pace. This is like a newspaper: the writer publishes once, unlimited readers can read any time.

■ **Simple Analogy:** Imagine a bakery putting a 'Fresh Bread' sign in the window. Any customer (service) who sees it can react — buy bread, take a photo, review it. The bakery (producer) doesn't call each customer individually.

Key Concepts

Event	A fact that happened: OrderPlaced, PaymentFailed, UserRegistered. Immutable — can't change it.
Producer	The service that publishes events. It does not know who reads them.
Consumer	A service that listens for events and reacts. Can be many consumers for one event.
Topic	A named channel in Kafka where events of one type are published.
Partition	A topic is split into partitions for parallelism. Events in one partition are ordered.
Outbox Pattern	Write event to DB table first (atomic with business data), then publish — guarantees no lost events.
Dead Letter Topic	Where events go when a consumer fails repeatedly — for manual inspection.

Diagram — Event-Driven Flow with Kafka

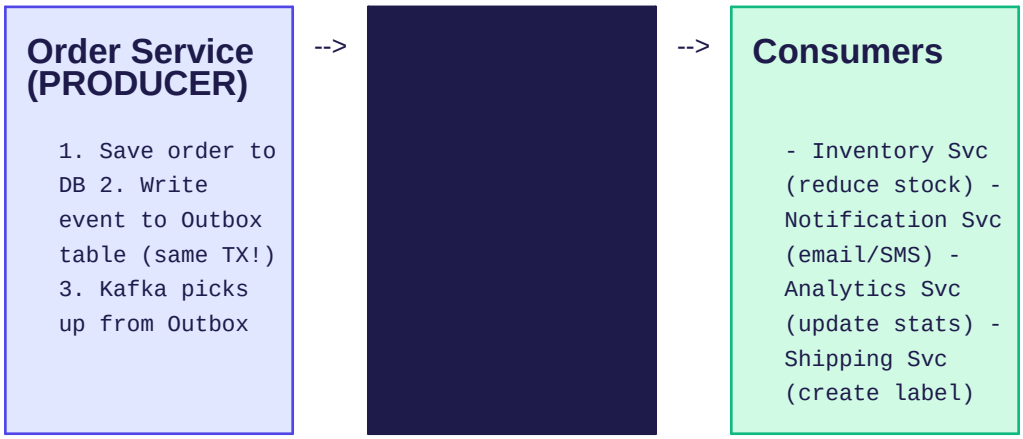

Figure 4: One producer, many consumers — all via Kafka topic

Event Types

Notification Event — Just says something happened. Consumer must call back for details.
Example: { "type": "ORDER_PLACED", "orderId": "123" }

Event-Carried State Transfer — Contains all the data consumers need.
Example: { "type": "ORDER_PLACED", "orderId": "123", "items": [...], "total": 99.99 }

Event Sourcing — The database IS the event log. Replay events to rebuild state.

Why Use Event-Driven Architecture?

- ■ Loose coupling — producer doesn't know its consumers exist
- ■ Independent scaling — each consumer scales separately
- ■ Temporal decoupling — consumer can be down; events wait in Kafka
- ■ Natural audit trail — every event is persisted and replayable
- ■ Easy to add new features — just add a new consumer

■ **Real-World Example:** Uber: When a rider books a trip, one TripRequested event is published to Kafka. Six services react independently: Driver Matching, Surge Pricing, Maps, Notifications, ETA, and Analytics — none of them call each other. Each team deploys independently.